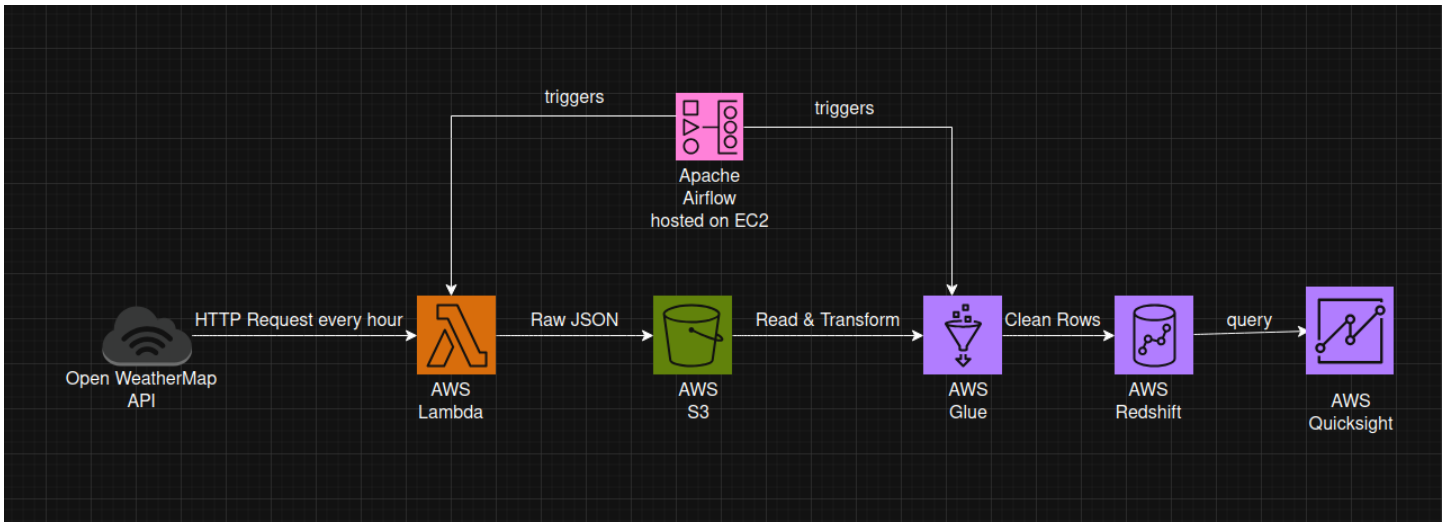
Real-Time Weather Data Pipeline

What Are You Actually Building?

OpenWeatherMap API
↓ (every hour)
AWS Lambda
← fetches weather data & saves raw JSON
↓
AWS S3
← stores raw data ("data lake")
↓
AWS Glue
← cleans & transforms the data
↓
AWS Redshift
← stores clean data ("data warehouse")
↓
Apache Airflow
← schedules & monitors the whole pipeline
↓
Dashboard
← visualises trends (QuickSight / Power BI)



Real-Time Weather Data Pipeline architecture

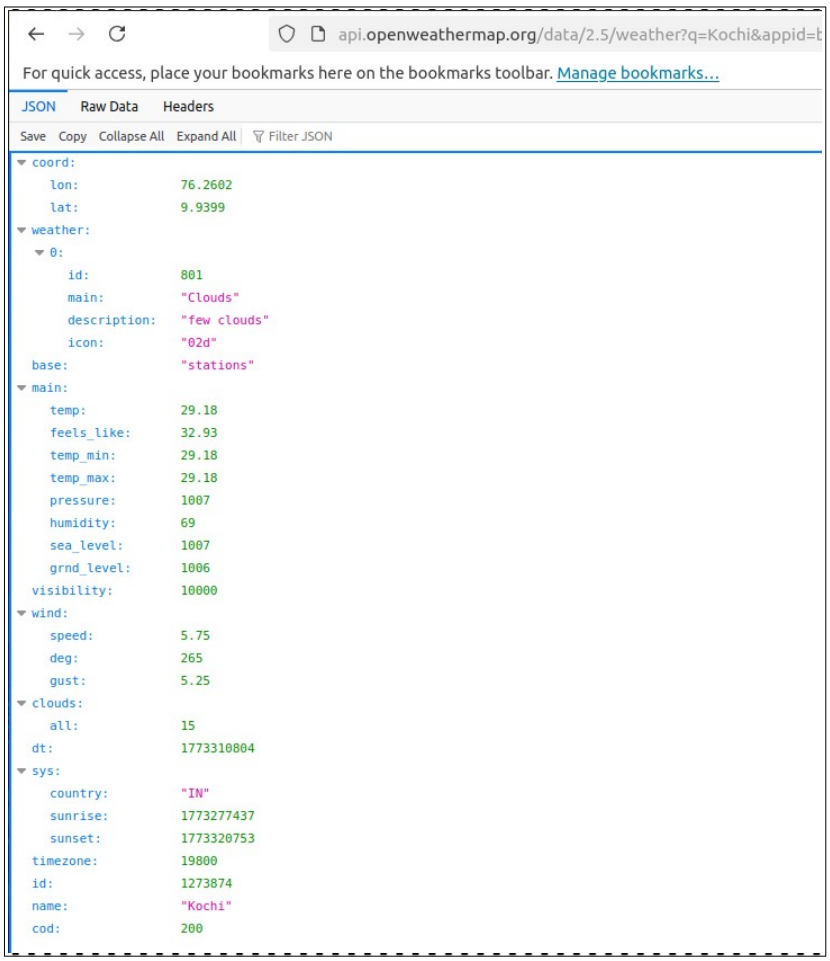
Phase 0 — Setup

What you're doing:
Creating free accounts for AWS and OpenWeatherMap

OpenWeatherMap → https://openweathermap.org/api → Sign up → Get your free API key (default key) from API key tab.

Run this in the browser
http://api.openweathermap.org/data/2.5/weather?q=Kochi&appid=<api_key>&units=metric

when it runs there will be an output which shows the details of city Kochi



AWS Free Tier → https://aws.amazon.com/free (if you don't have one)
→Create an IAM user and use it instead of root user (recommended)

Phase 1 — Data Ingestion with AWS Lambda

What you're doing:
Writing a Python function that calls the weather API and drops the response into S3 as a JSON file, every hour.

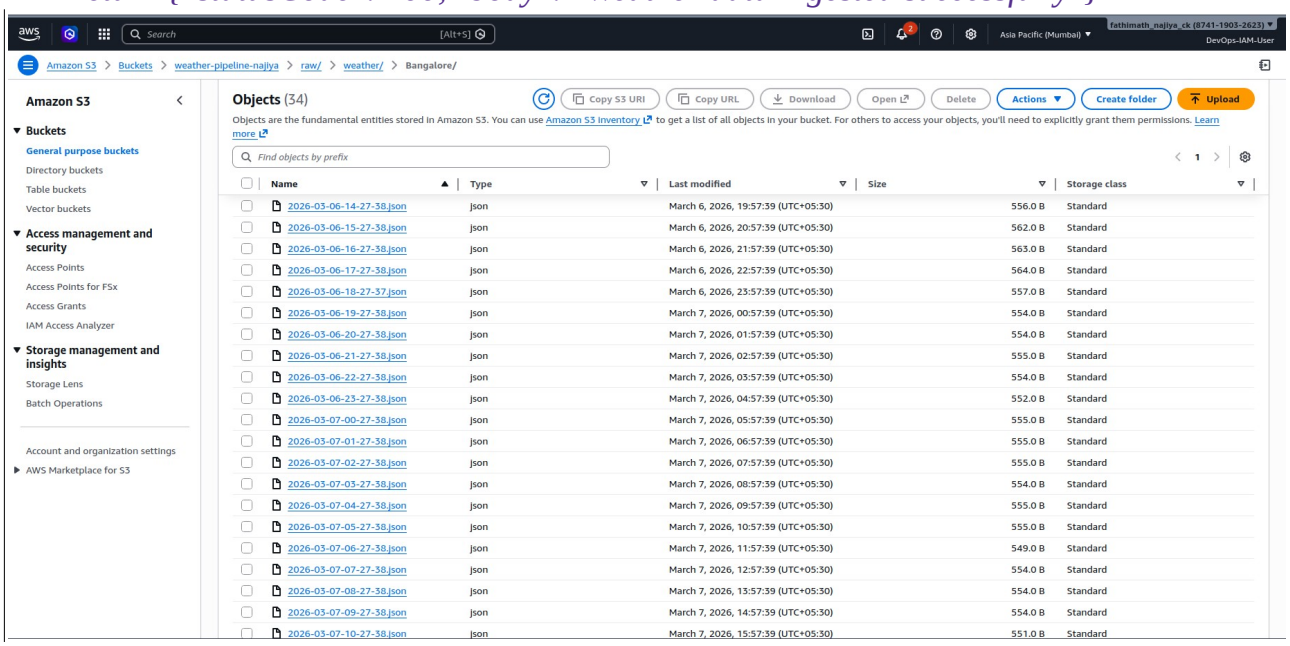
Create your S3 Bucket
• Go to AWS Console → S3 → Create Bucket
• Name it as weather-pipeline-najiya
• Region: ap-south-1 (Mumbai, closest to Kerala)
• Keep all other settings default
• Remember: S3 bucket names are globally unique across all AWS accounts

Deploy Lambda
1. AWS Console → Lambda → Create Function
2. Runtime: Python 3.14
3. Write a python code to fetch the data from Open WeatherMap API and store the raw data into the S3 bucket that I created.
4. Add environment variables for API_KEY and BUCKET_NAME (never hardcode secrets! Into the lambda code directly)
5. Give Lambda an IAM Role with AmazonS3FullAccess
6. Test it — check your S3 bucket for the JSON files

<u>Lambda code (lambda_function.py)</u>
weather-lambda-function code:

```
import json
import boto3
import os
import urllib.request
from datetime import datetime

API_KEY = os.environ["API_KEY"]
BUCKET_NAME = os.environ["BUCKET_NAME"]
CITIES = ["Kochi", "Bangalore", "Mumbai", "Delhi"]


s3_client = boto3.client("s3")
```

```
def lambda_handler(event,context):
    for city in CITIES:
        url = f"http://api.openweathermap.org/data/2.5/weather?q={city}&appid={API_KEY}
&units=metric"

        with urllib.request.urlopen(url) as response:
            data = json.loads(response.read().decode())

        timestamp = datetime.utcnow().strftime("%Y-%m-%d-%H-%M-%S")
        file_key = f"raw/weather/{city}/{timestamp}.json"

        s3_client.put_object(
            Bucket = BUCKET_NAME,
            Key = file_key,
            Body = json.dumps(data),
            ContentType="application/json"
        )
    return {"statusCode": 200, "body": "Weather data ingested successfully"}
```



To verify the data inside the file

click one of the file → click open →we can see the data in that file in our browser

Phase 2 — Transformation with AWS Glue

What you're doing:

Glue reads the messy raw JSONs from S3, extracts only the fields you need, and loads clean rows into Redshift.

S3 (raw JSON) → Glue (PySpark) → Redshift (clean table)

What is PySpark?

PySpark is Python for big data. Instead of processing one file at a time, Spark processes thousands of files in parallel. Glue uses PySpark under the hood.

Create Redshift table first

Go to AWS Console → Redshift → Query Editor and run this SQL:

```
CREATE TABLE IF NOT EXISTS weather_data (
city
VARCHAR(100),
country
VARCHAR(10),
temperature FLOAT,
feels_like
humidity
wind_speed
FLOAT,
INT,
```

FLOAT,
description VARCHAR(200),
recorded_at TIMESTAMP
);

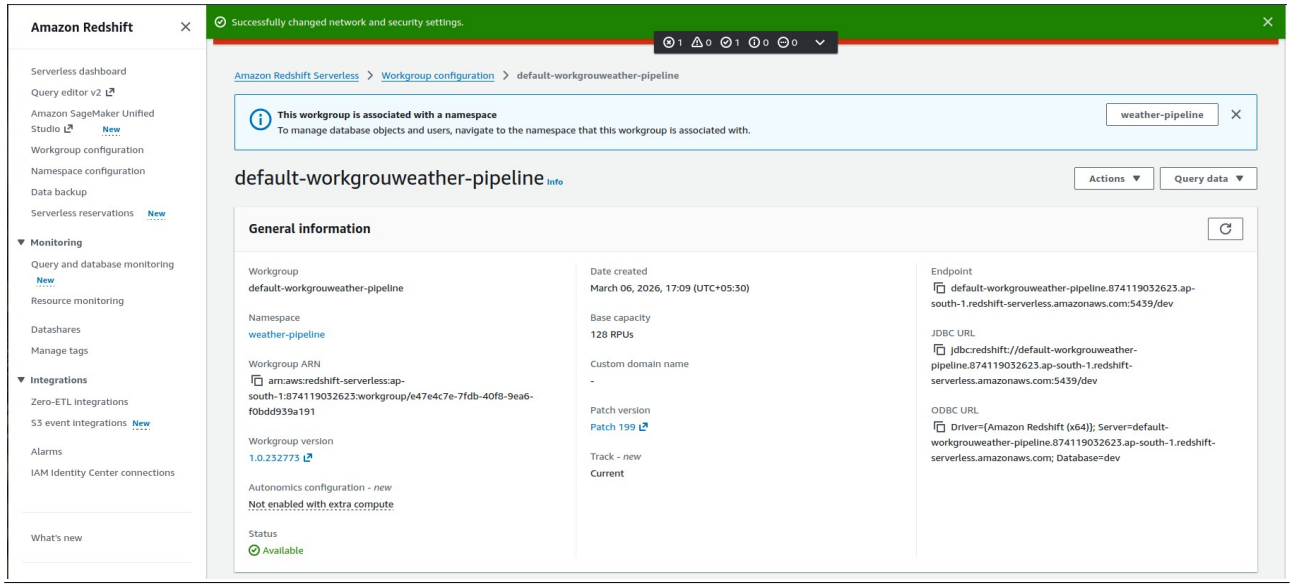
Before writing the Glue script, you need 2 things ready:

Get your Redshift Serverless endpoint

1. Go to **Redshift Serverless** → Workgroups → your workgroup
2. Copy the **Endpoint** — default-workgrouweather-pipeline.874119032623.ap-south-1.redshift-serverless.amazonaws.com:5439/dev

Create an IAM role

1. AWS Console → IAM → Roles → Create Role
2. Trusted entity: Glue
3. Attach these 2 policies:
   - AmazonS3FullAccess
   - AmazonRedshiftFullAccess
4. Name it: glue-weather-role



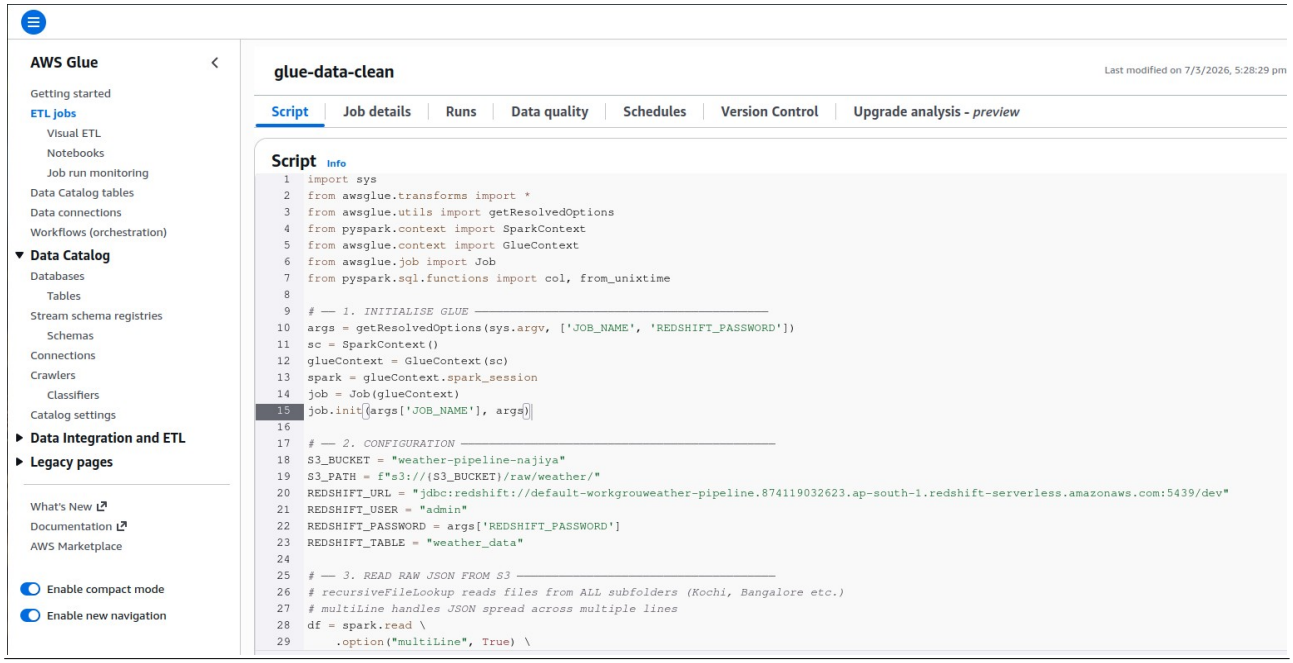
Set up Glue

1. AWS Console → Glue → Create Job
2. Type: Spark script editor
3.Write a Pyspark script to clean the raw data and store directly into Redshift(using *format("jdbc")*).
4. Add environmental parameters in job details tab for Redshift connection string, bucket name, so that I don't need to hard code it.
5. Create a Glue IAM Role with S3 + Redshift access
6. Run it and watch the logs!

<u>glue-data-clean code:</u>

```
import sys
from awsglue.transforms import *
from awsglue.utils import getResolvedOptions
from pyspark.context import SparkContext
from awsglue.context import GlueContext
from awsglue.job import Job
from pyspark.sql.functions import col, from_unixtime

# ── 1. INITIALISE GLUE ──────────────────────────────────────────────────────
args = getResolvedOptions(sys.argv, ['JOB_NAME', 'REDSHIFT_PASSWORD'])
sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session
job = Job(glueContext)
job.init(args['JOB_NAME'], args)

# ── 2. CONFIGURATION ────────────────────────────────────────────────────────
S3_BUCKET = "weather-pipeline-najiya"
S3_PATH = f"s3://{S3_BUCKET}/raw/weather/"
REDSHIFT_URL = "jdbc:redshift://default-workgrouweather-pipeline.874119032623.ap-south-1.redshift-
serverless.amazonaws.com:5439/dev"
REDSHIFT_USER = "admin"
REDSHIFT_PASSWORD = args['REDSHIFT_PASSWORD']
REDSHIFT_TABLE = "weather_data"

# ── 3. READ RAW JSON FROM S3 ────────────────────────────────────────────────
# recursiveFileLookup reads files from ALL subfolders (Kochi, Bangalore etc.)
# multiLine handles JSON spread across multiple lines
df = spark.read \
    .option("multiLine", True) \
    .option("recursiveFileLookup", True) \
    .json(S3_PATH)

# ── 4. EXTRACT ONLY THE FIELDS YOU NEED ─────────────────────────────────────
transformed_df = df.select(
    col("name").alias("city"),
    col("sys.country").alias("country"),
    col("main.temp").alias("temperature"),
    col("main.feels_like").alias("feels_like"),
    col("main.humidity").alias("humidity"),
```

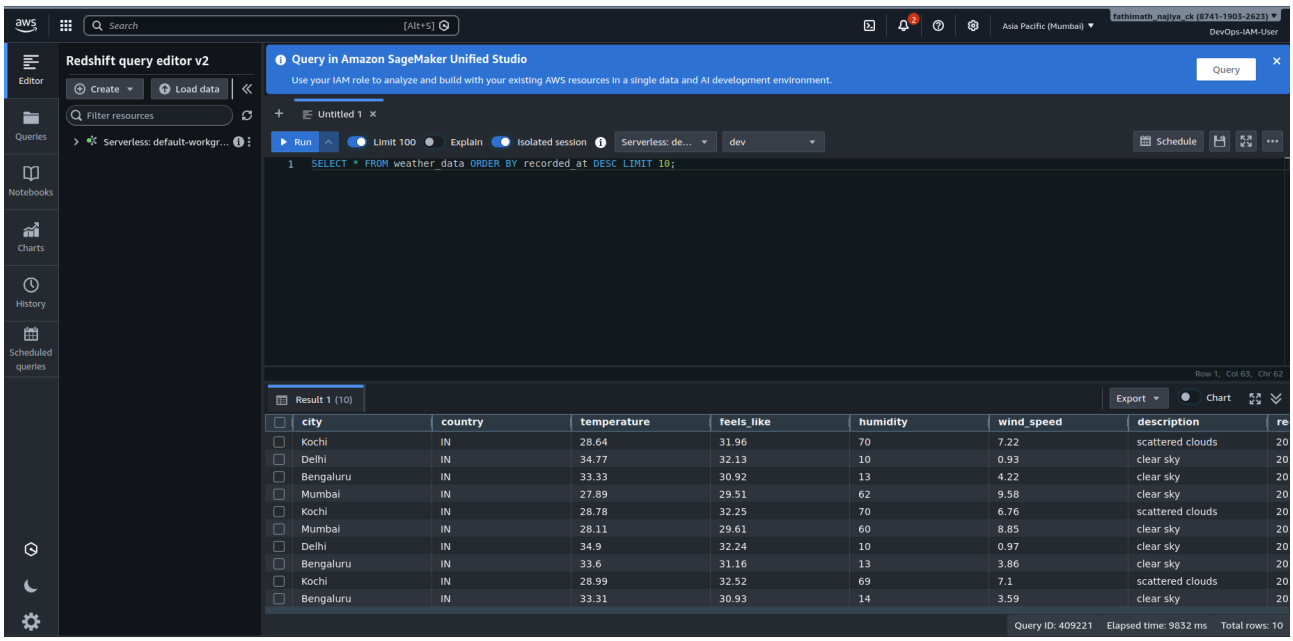
```
    col("wind.speed").alias("wind_speed"),
    col("weather")[0]["description"].alias("description"),
    from_unixtime(col("dt")).alias("recorded_at")
)

# ── 5. WRITE CLEAN DATA TO REDSHIFT ──────────────────────────────────────
transformed_df.write \
    .format("jdbc") \
    .option("url", REDSHIFT_URL) \
    .option("dbtable", REDSHIFT_TABLE) \
    .option("user", REDSHIFT_USER) \
    .option("password", REDSHIFT_PASSWORD) \
    .option("driver", "com.amazon.redshift.jdbc42.Driver") \
    .mode("append") \
    .save()

# ── 6. CLOSE THE JOB ─────────────────────────────────────────────────────
job.commit()
```



Verify in Redshift

Verify by writing the query in redshift's query v2 option.

```
SELECT * FROM weather_data ORDER BY recorded_at DESC LIMIT 10;
```



Phase 3 — Orchestration with Apache Airflow

What you're doing:
Setting up Airflow to automatically run Lambda → Glue every hour, in the right order, and alert you if something fails.

<u>The most important concept: DAGs</u>
DAG = Directed Acyclic Graph. It's just a fancy word for: "a set of tasks with a defined order, no loops."

ingest_weather_data → transform_and_load
(Lambda) → (Glue Job)

Launch EC2 Instance:

1. AWS Console → **EC2** → **Launch Instance**
2. Settings:
   - **Name:** airflow-server
   - **AMI:** Amazon Linux 2023
   - **Instance type:** t2.medium ⚠ (Airflow needs more memory than t2.micro)
   - **Key pair:** created a new key pair → save the .pem file (file name : airflow.pem)

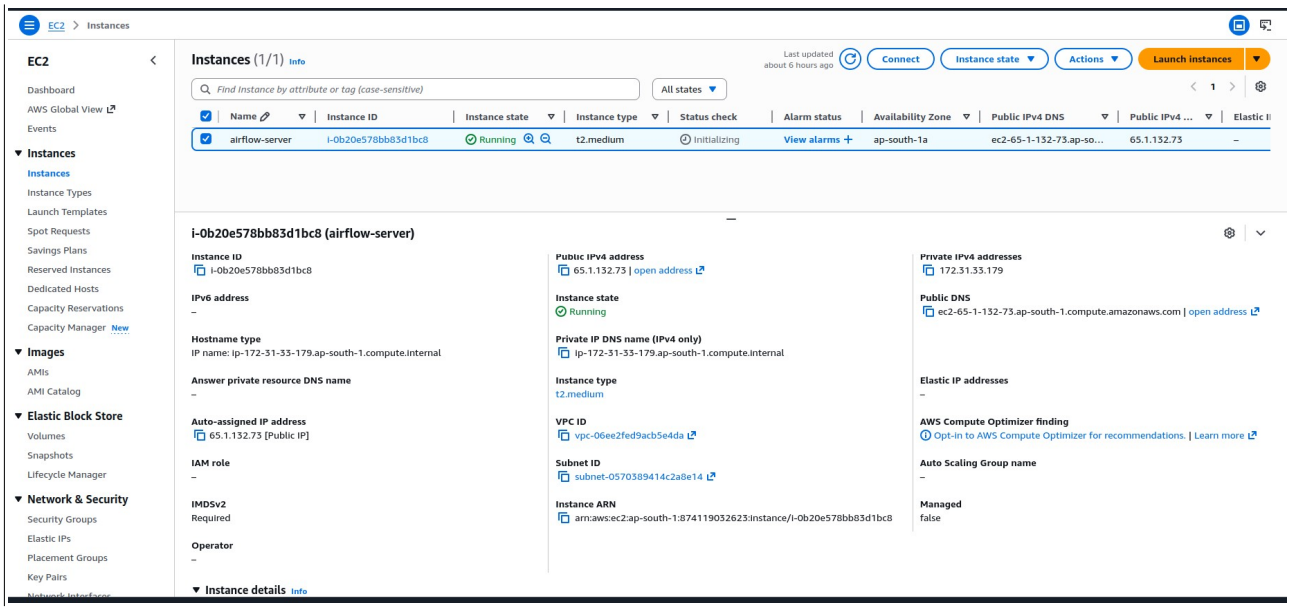
- **Security group:** allow **SSH (port 22)** and **Custom TCP (port 8080)** from anywhere 0.0.0.0/0

3. Click **Launch**

steps to do in terminal

SSH into your instance:

ssh -i airflow.pem ec2-user@ec2-65-1-132-73.ap-south-1.compute.amazonaws.com



Install Airflow:

Update system
sudo dnf update -y

Install pip
sudo dnf install python3-pip -y

Install virtualenv (best practice)
pip3 install virtualenv
python3 -m virtualenv airflow_env
source airflow_env/bin/activate

Set Airflow home
export AIRFLOW_HOME=~/airflow

Install Airflow with AWS providers
pip install "apache-airflow==2.7.0" apache-airflow-providers-amazon

Initialise Airflow database
airflow db init

Create admin user
```
airflow users create \
  --username admin \
  --password admin \
  --firstname Najiya \
  --lastname Nazheer \
  --role Admin \
  --email you@email.com
```

Start Airflow
airflow webserver --port 8080 & airflow scheduler &

Step 3 — Access Airflow UI: Open in your browser:
http://ec2-65-1-132-73.ap-south-1.compute.amazonaws.com:8080

⚠ **One important thing —** every time you SSH back into EC2 after closing the session,

1.First you need to reactivate the virtual environment:
   source airflow_env/bin/activate
   export AIRFLOW_HOME=~/airflow
2.Check if Airflow is still running: *ps aux | grep airflow*
3.If you see airflow processes → Airflow is still running in background, just go straight to creating the DAG file.
   3.1 Create dags folder in EC2 terminal: *mkdir ~/airflow/dags*
   3.2 Create the DAG file: *vi ~/airflow/dags/weather_dag.py*
   3.3 write this code in that vim editor and save it:

```
from airflow import DAG
from airflow.providers.amazon.aws.operators.lambda_function import LambdaInvokeFunctionOperator
from airflow.providers.amazon.aws.operators.glue import GlueJobOperator
from datetime import datetime, timedelta

# ── DEFAULT SETTINGS FOR ALL TASKS ──────────────────────────────────────
default_args = {
    'owner': 'najiya',
    'retries': 2,                    # retry 2 times if task fails
    'retry_delay': timedelta(minutes=5),    # wait 5 mins between retries
    'email_on_failure': False,
}
```

```
# ── DAG DEFINITION ──────────────────────────────────────────────────────────
with DAG(
    dag_id='weather_pipeline',              # name shown in Airflow UI
    default_args=default_args,
    description='Hourly weather data pipeline',
    schedule_interval='@hourly',            # runs every hour automatically
    start_date=datetime(2026, 3, 7),        # today's date
    catchup=False,                          # don't run missed past schedules
) as dag:

    # ── TASK 1: RUN LAMBDA ──────────────────────────────────────────────────
    ingest_weather_data = LambdaInvokeFunctionOperator(
        task_id='ingest_weather_data',
        function_name='weather-lambda-function',
        aws_conn_id='aws_default',
    )

    # ── TASK 2: RUN GLUE ────────────────────────────────────────────────────
    transform_and_load = GlueJobOperator(
        task_id='transform_and_load',
        job_name='clean-glue-job',
        aws_conn_id='aws_default',
        region_name='ap-south-1',
       iam_role_name='glue-weather-role',
    )

    # ── PIPELINE ORDER ──────────────────────────────────────────────────────
    # Lambda runs first, then Glue runs after Lambda succeeds
    ingest_weather_data >> transform_and_load
```

*Now let's set up the **AWS connection** in Airflow so it can talk to Lambda and Glue.*

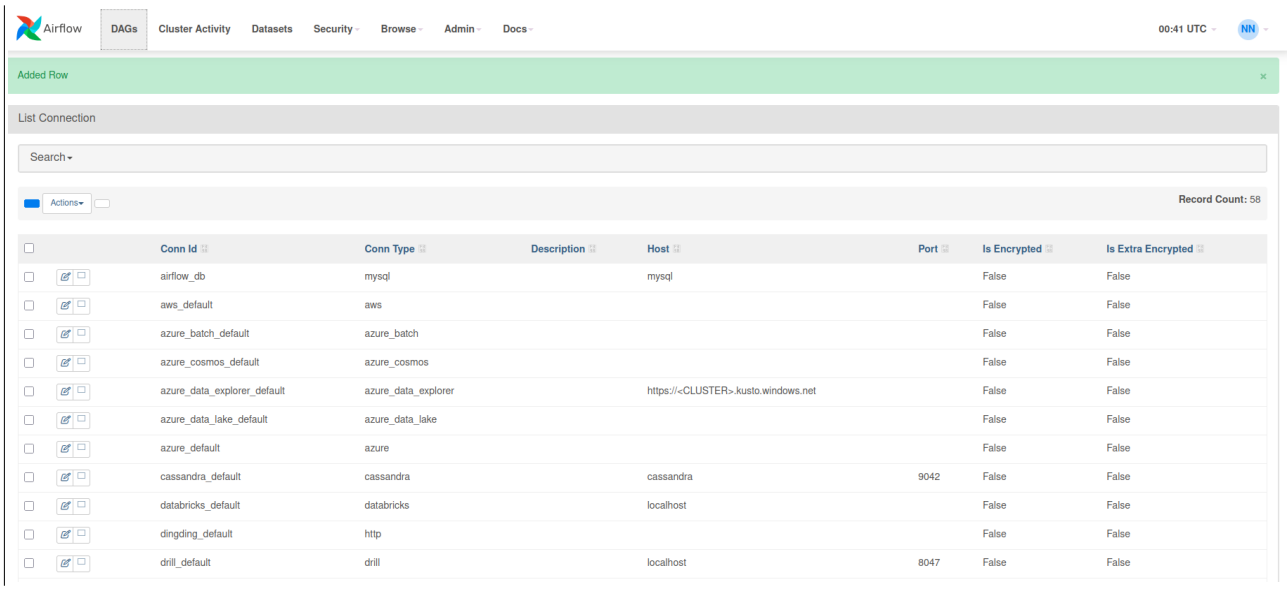
In Airflow UI:

1. Click **Admin** in top menu
2. Click **Connections**
3. Click `"+"` to add new connection
4. Fill in:
    - **Connection Id:** aws_default
    - **Connection Type:** Amazon Web Services
    - **AWS Access Key ID:** your AWS access key
    - **AWS Secret Access Key:** your AWS secret key
    - **Extra:** {"region_name": "ap-south-1"}

To get AWS Access Key ID ans Secret Key ID

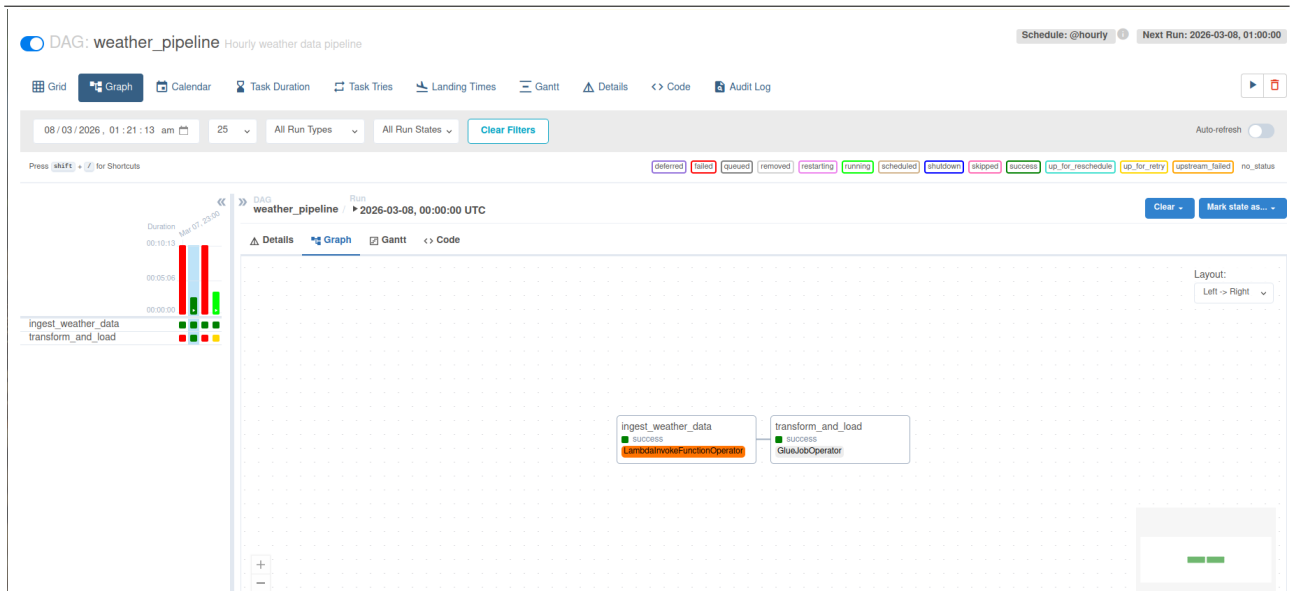
1. AWS Console → **IAM** → **Users** → **your user**
2. Click **Security credentials** tab

3. Click **"Create access key"**
4. Copy both **Access Key ID** and **Secret Access Key** — save them somewhere safe!



Now trigger the DAG:

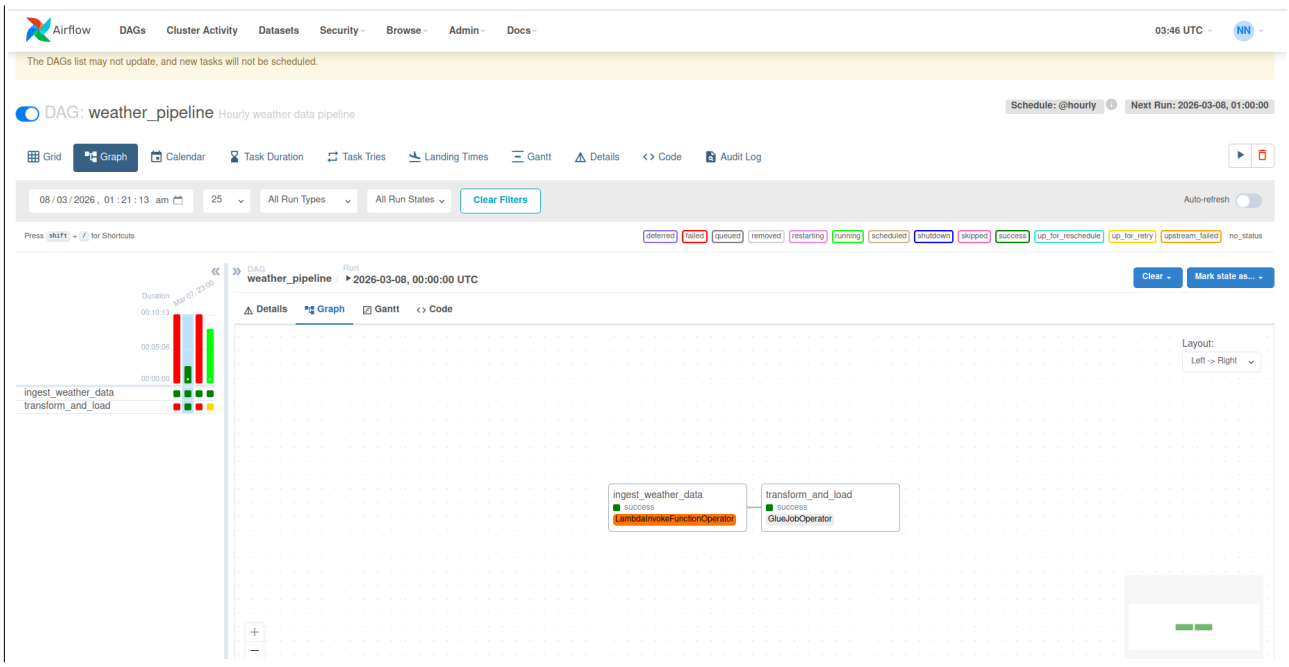
1. In Airflow UI → click **weather_pipeline**
2. Toggle the **switch on the left** from OFF to ON
3. Click **play button**
4. Click **"Trigger DAG"**
5. Watch the **Graph view** — both tasks should turn green



Phase 4 — Dashboard

QuickSight

1. AWS Console → QuickSight → Connect to Redshift
2. Create a dataset from your weather_data table
3. Build visuals: temperature trends, humidity by city, wind speed over time



<u>PROJECT OVERVIEW:</u>

Built an end-to-end real-time data pipeline on AWS ingesting weather data from a public API for 4 Indian cities. Used Lambda for serverless ingestion, S3 as a data lake, Glue/PySpark for transformation, and Redshift as the data warehouse. Orchestrated with Apache Airflow on EC2. Visualised trends in a QuickSight dashboard.